

命令の詳細

蓬萊 尚幸

I 形式の immediate の取り扱い

I 形式命令の immediate 部は 16 ビットであるが、ID ステージにおいて、32 ビットに変換する。変換後の 32 ビットデータを data とする。変換方法には、immediate 符号拡張 (SignExtImm)、ゼロ拡張 (ZeroExtImm)、分岐アドレス (BranchAddr) の 3 種類が存在する。

SignExtImm

data の下位 16 ビット (data[0~15]) は immediate フィールドの値となる。data の上位 16 ビットは immediate の符号ビットである最上位ビット (immediate[15]) で埋める。すなわち、data[16~31] は FFFF_{16} か 0 となる。

ZeroExtImm

data の下位 16 ビット (data[0~15]) は immediate フィールドの値となる。data の上位 16 ビットは、常に、0 で埋める。すなわち、data[16~31] = 0 である。

BranchAddr

MIPS アーキテクチャでは命令はすべて 32 ビットデータとして扱われ、メモリ上では 4 の倍数のアドレスに配置される。そこで、immediate フィールドは 4 バイト単位で数えたアドレス値を格納する。また、分岐命令では、アドレスを PC と相対的に表すので、immediate フィールドの値は、現時点より前への分岐ならば負、後ろへ分岐ならば正となる。よって、immediate フィールドの値を 2 ビット左にシフトし、符号拡張する。すなわち、data[0~1] = 0, data[2~17] = immediate であり、data[18~31] は、immediate[15] が 0 の場合は 0 になり、immediate[15] が 1 の場合は 3FFF_{16} になる。

J 形式の addr の取り扱い

I 形式の immediate フィールドの BranchAddr と同様に、J 形式の addr フィールドは 4 バイト単位である。addr フィールドはアドレスそのものなので符号無しである。よって、実効アドレス Addr(32 ビット) は、addr フィールド(26 ビット)の値を 2 ビット左にシフトし、ゼロ拡張した値である。すなわち、Addr[0~1] = 0, Addr[2~27] = addr, Addr[28~31] = 0 になる。J 形式で指定可能な実効アドレスは、 $\text{00000000}_{16} \sim \text{0FFFFFFF}_{16}$ の範囲のみであること

に注意せよ。

命令内の未使用フィールド

add 命令の shamt フィールドや jr 命令の rt, rd, shmat フィールドなどの未使用フィールドの値は、0 にする慣習になっている。ただし、0 以外の値を使っても、(ハードウェア的に) 無視されるだけであり、動作は変わらない。

即値符号無し演算

addiu, subiu, sltiu では、まず、immediate フィールドを SignExtImm 変換で 32 ビット値に符号拡張し、次に、符号無し整数として演算を実行する。

addiu, subiu は addi, subi と演算結果は同じである。違いはオーバーフローの例外の発生の有無だけである。

sltiu と slti は演算結果が大きく異なる。Immediate フィールドが FFFF_{16} である場合、SignExtImm 変換で得られる 32 ビット値は FFFFFFF_{16} となる。slti では、この値を符号付き整数 -1 としてレジスタと符号付き整数比較される。一方、sltiu では、整数 $2^{16} - 1$ (大きな数) としてレジスタと符号無し整数比較される。

乗除算と Hi レジスタと Lo レジスタ

原理的に 32 ビット値同士の乗算結果は 64 ビットとなる。そこで、この計算結果を格納するために 32 ビットレジスタ 2 個、Hi レジスタと Lo レジスタを用意する。乗算結果の上位 32 ビットは Hi レジスタに格納され、下位 32 ビットは Lo レジスタに格納される。

原理的に 32 ビット値同士の除算結果である商と余りはともに 32 ビットとなる。商は Lo レジスタに格納され、余りは Hi レジスタに格納される。

Hi レジスタ、Lo レジスタの値をレジスタに転送する命令 mfhi, mflo が用意されている。

浮動点数の取り扱い

MIPS アーキテクチャでは、CPU に浮動点数演算を実装されていない。そこで、浮動点数演算は、コプロセッサ 1 (CP1) を用いる。CP1 とのデータ交換は、32 個の 32 ビット浮動点数レジスタ (番号は 0~31、アセンブリ言語では $\$f0 \sim \$f31$) を介して行う。単精度浮動点数の交換には浮動点レジスタ 1 個を用いる。倍精度浮動点数の交換には偶数番号と次の奇数番号の浮動点数レジスタ 2 個 (例えば、 $\$f0$ と $\$f1$ 、 $\$f2$ と $\$f3$) を用いる。64 ビット倍精度浮動点数の上位 32 ビットは偶数番号の浮動点数レジスタに格納され、下位 32 ビットは奇数番

号の浮動点数レジスタに格納される。

浮動点数命令は、四則計算命令 8 個(4[加減乗除]×2[単・倍精度])、メモリと浮動点数レジスタ間のデータ転送命令 4 個 (2[ロード・ストア]×2[単・倍精度])、比較命令 6 個 (3[>≧]×2[単・倍精度 2])、分岐命令 2 個が実装されている。

比較命令の結果は、1 ビット FPcond レジスタに格納される。比較した結果が真ならば 1 が格納され、偽ならば 0 が格納される。分岐命令では PFcond の値が用いられ、PFcond の値が 0 で分岐する bc1f 命令と 1 で分岐する bc1t 命令の 2 個である。

排他制御

複数の CPU やコアを用いて並列処理する場合、排他制御が必要である。MIPS アーキテクチャでは、排他制御のためのプリミティブとして、メモリに対するテスト&セット命令が用意されている。

ll 命令では、lw 命令と同様にメモリからデータをロードし、実効アドレスに対して「リンク状態」を設定する。

sc 命令では、直前の ll 命令で設定したリンク状態がまだ有効であり、直前の ll 命令の実行の後に、他の CPU やコアが同じアドレスに書き込みをしていないときに限って、sw 命令と同様にメモリに汎用レジスタの値をストアする。逆に、他の CPU やコアが同じアドレスに書き込んでいたり、割込みなどによりリンク状態が失われたりキャッシュの変化により無効化されていた場合は、メモリへのストアは実行しない。さらに、ストアに成功した場合は汎用レジスタに 1 を格納し、失敗した場合は 0 を格納する。

例外処理

MIPS アーキテクチャでは、例外や割込みの管理をコプロセッサ 0(CP0)で行う。例外や割込みの管理以外にも、CP0 は、仮想記憶における TLB の管理なども行う。mfc0 命令を用いて CP0 内のレジスタの値を読み込むことができる。たとえば、レジスタ 13(\$13)は直前に発生した例外の種類、レジスタ 14(\$14)は例外発生時のプログラムカウンタ、レジスタ 8(\$8)は不正メモリアクセス時にアクセスしたメモリのアドレスが格納される。

R 形式：op rd,rs,rt

op = add, addu, and, nor, or, slt, sltu, sub, subu.

rs と rt で指定された汎用レジスタの値を用いて演算した結果を rd で指定された汎用レジスタに格納する。

R 形式： *op* rd,rt,shamt

op = sll, sra, srl.

rt で指定された汎用レジスタの値を shamt だけシフトした値を rd で指定された汎用レジスタに格納する。

R 形式： *op* rs

op = jr のみ。

rs で指定された汎用レジスタの値をプログラムカウンタに格納する。

R 形式： *op* rs,rt

op = div, divu, mult, multu.

rs と rt で指定された汎用レジスタの値を用いて演算した結果を Hi レジスタと Lo レジスタに格納する。

R 形式： *op* rd

op = mfhi, mflo.

mfhi では Hi レジスタの値、mflo では Lo レジスタの値を rd で指定した汎用レジスタに格納する。

R 形式： *op* rd,rs

op = mfc0 のみ。

rs で指定された制御レジスタの値(CR[rs])を rd で指定された汎用レジスタに格納する。

I 形式： *op* rt,immediate(rs)

op = lbu, lhu, ll, lw, sb, sc, sh, sw.

lbu, lhu, ll, lw では、rs レジスタの値と immediate の値を SignExtImm 変換した結果を足して得られる値を実効アドレスとしてメモリからロードした値を rt で指定された汎用レジスタに格納する。第 1 オペランドが rt であることに注意。

sb, sc, sh, sw では、rt レジスタの値と immediate の値を SignExtImm 変換した結果を足して得られる値を実効アドレスとして rs で指定された汎用レジスタの値をメモリにストアす

る。

ただし、sc では、ストアに失敗する場合がある(前述の「排他制御」を参照)。ストアに成功した場合は rt に 1 を格納し、失敗した場合は rt に 0 を格納する。

I 形式： *op* rt,rs,immediate

op = addi, addiu, andi, beq, bne, ori, slti, sltiu.

addi, addiu, slti, sltiu では immediate の値を SignExtImm 変換した結果と rs で指定された汎用レジスタの値を演算した結果を rt で指定された汎用レジスタに格納する。

andi, ori では immediate の値を ZeroExtImm 変換した結果と rs で指定された汎用レジスタの値を演算した結果を rt で指定された汎用レジスタに格納する。

beq, bne では、rt と rs で指定された汎用レジスタの値を用いて条件判定を行い、結果が真ならば immediate の値を BranchAddr 変換した結果とプログラムカウンタ+4(次の命令が格納されているアドレス)を足した結果をプログラムカウンタに格納する。判定結果が偽ならば何もしない(次の命令に進む)。SignExtImm 変換した結果が負ならば後方に進み(戻り)、正ならば先方に進む。

I 形式： *op* rt,immediate

op = lui のみ.

immediate の値(16 ビット)を 16 ビット左シフトした値(下位 16 ビットは 0 になる)を rt で指定されたレジスタに格納する。

I 形式： *op* ft,immediate(rs)

op = ldc1, lwcl, sdc1, swcl.

rs レジスタの値と immediate の値を SignExtImm 変換した結果を実効アドレスとする。

ldc1 では、実効アドレスからの 8 バイトをロードした値を ft で指定された浮動数点レジスタに格納する。

sdcl では、実効アドレスからの 4 バイトに ft で指定された浮動数点レジスタの値をストアする。

ldc1 では、実効アドレスからの 8 バイトをロードした値(64 ビット)の上位 32 ビットを ft で指定された浮動数点レジスタに格納し、下位 32 ビットを ft で指定された浮動点レジスタの次の浮動点レジスタに格納する。

sdcl では、ft で指定された浮動数点レジスタの値を実効アドレスからの 4 バイトに ft で指定された浮動点レジスタの値をストアし、ft で指定された浮動点レジスタの次の浮

動点数レジスタの値を実効アドレス+4からの4バイトにストアする。ロード命令もストア命令も、第1オペランドがftであることに注意。ftはI形式のrtフィールドに格納する。

J 形式： *op* addr

op = j, jal.

addr を2ビット左シフトした値をプログラムカウンタに格納する。jal では、先に汎用レジスタ\$ra にプログラムカウンタ+2を格納した後に、上記のプログラムカウンタへの格納を実行する。

FR 形式： *op* fd,fs,ft

op = add.d, add.s, div.d, div.s, mul.d, mul.s, sub.d, sub.s.

add.s, div.s, mul.s, sub.s では、fs と ft で指定された浮動数点レジスタの値を用いて単精度浮動点数演算の結果を fd で指定された浮動点数レジスタに格納する。

add.d, div.d, mul.d, sub.d では、fs で指定された浮動数点レジスタの値を上位32ビットとしその次のレジスタの値を下位32ビットにした64ビット値と ft で指定された浮動数点レジスタの値を上位32ビットとしその次のレジスタの値を下位32ビットにした64ビット値を用いて倍精度浮動点演算の結果の上位32ビットを fd で指定された浮動点数レジスタに格納し下位32ビットをその次の浮動点数レジスタに格納する。

FR 形式： *op* fs,ft

op = c.eq.d, c.lt.d, c.le.d, c.eq.s, c.lt.s, c.le.s.

c.eq.s, c.lt.s, c.le.s では、fs と ft で指定された浮動数点レジスタの値を用いて単精度浮動点数比較する。

c.eq.d, c.lt.d, c.le.d では、fs で指定された浮動数点レジスタの値を上位32ビットとしその次のレジスタの値を下位32ビットにした64ビット値と ft で指定された浮動数点レジスタの値を上位32ビットとしその次のレジスタの値を下位32ビットにした64ビット値を用いて倍精度浮動点比較する。比較結果はFPcondレジスタに格納される。

FI 形式： *op* immediate

op = bclf, bc1t.

FPcondレジスタの値が1ならば、beq, bneと同様に、immediateの値をBranchAddr変換した結果とプログラムカウンタ+4(次の命令が格納されているアドレス)を足した結果を

プログラムカウンタに格納する。FPcond レジスタの値が 0 ならば、何もしない(次の命令に進む)。

以 上